

Lecture 05 - Clipping

This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Source chunk	course_text_parts/03_lectures/05_clipping.txt
Extracted slide pages	56
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Clipping Projected Geometry 1/2
Source cue	- Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized

Professor-style explanation	The slide 'Clipping Projected Geometry 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Clipping Projected Geometry 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Clipping Projected Geometry 1/2'?

Page 3	Clipping Projected Geometry 2/2
Source cue	- Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping
Professor-style explanation	The slide 'Clipping Projected Geometry 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Clipping Projected Geometry 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Clipping Projected Geometry 2/2'?

Page 4	5.1 Cohen-Sutherland Line Clipping
Source cue	5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping
Professor-style explanation	The slide '5.1 Cohen-Sutherland Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.1 Cohen-Sutherland Line Clipping'?

Page 5	5.1 Cohen-Sutherland Line Clipping
Source cue	Clip lines against rectangle

Professor-style explanation	The slide '5.1 Cohen-Sutherland Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Clip lines against rectangle. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Clip lines against rectangle
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.1 Cohen-Sutherland Line Clipping'?

Page 6	Brute-Force Line Clipping 1/2
Source cue	- Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary
Professor-style explanation	The slide 'Brute-Force Line Clipping 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Brute-Force Line Clipping 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Brute-Force Line Clipping 1/2'?
Page 7	Brute-Force Line Clipping 2/2
Source cue	- Using the parameter representation for lines $x = x_0 + t(x_1 - x_0)$ / $y = y_0 + t(y_1 - y_0)$ / $t = (x - x_0) / (x_1 - x_0)$ / $t = (y - y_0) / (y_1 - y_0)$ / x_0, x_1, y_0, y_1 are the coordinates of the endpoints of the line segment.

Professor-style explanation	The slide 'Brute-Force Line Clipping 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Using the parameter representation for lines $x - x_0 = t(x_1 - x_0)$ / $y - y_0 = t(y_1 - y_0)$ / $0 \leq t \leq 1$ / $\mathbf{p} = \mathbf{p}_0 + t(\mathbf{p}_1 - \mathbf{p}_0)$. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Brute-Force Line Clipping 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Using the parameter representation for lines $x - x_0 = t(x_1 - x_0)$ / $y - y_0 = t(y_1 - y_0)$ / $0 \leq t \leq 1$ / $\mathbf{p} = \mathbf{p}_0 + t(\mathbf{p}_1 - \mathbf{p}_0)$.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Brute-Force Line Clipping 2/2'?
Page 8	Endpoint Analysis 1/2
Source cue	- Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $\mathbf{p} = (\mathbf{p}_0, \mathbf{p}_1)$, $\mathbf{p}_0 = (x_0, y_0)$ and $\mathbf{p}_1 = (x_1, y_1)$
Professor-style explanation	The slide 'Endpoint Analysis 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $\mathbf{p} = (\mathbf{p}_0, \mathbf{p}_1)$, $\mathbf{p}_0 = (x_0, y_0)$ and $\mathbf{p}_1 = (x_1, y_1)$. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Endpoint Analysis 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $\mathbf{p} = (\mathbf{p}_0, \mathbf{p}_1)$, $\mathbf{p}_0 = (x_0, y_0)$ and $\mathbf{p}_1 = (x_1, y_1)$
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 1/2'?

Page 9	Endpoint Analysis 2/2
Source cue	- Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / $x_{min} \leq x \leq x_{max}$ / and above y and below y , they are inside / $y_{min} \leq y \leq y_{max}$
Professor-style explanation	The slide 'Endpoint Analysis 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / $x_{min} \leq x \leq x_{max}$ / and above y and below y , they are inside / $y_{min} \leq y \leq y_{max}$. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Endpoint Analysis 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / $x_{min} \leq x \leq x_{max}$ / and above y and below y , they are inside / $y_{min} \leq y \leq y_{max}$
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 2/2'?
Page 10	Binary Coding of Half-Planes 1/3
Source cue	- Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode
Professor-style explanation	The slide 'Binary Coding of Half-Planes 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Binary Coding of Half-Planes 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Binary Coding of Half-Planes 1/3' into a causal sentence instead of repeating the slide title?
Page 11	Binary Coding of Half-Planes 2/3
Source cue	- Endpoint classification / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y) / 0\ 1$ / - Outcodes: / 1001 1000 1010
Professor-style explanation	The slide 'Binary Coding of Half-Planes 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Endpoint classification / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y) / 0\ 1$ / - Outcodes: / 1001 1000 1010. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Binary Coding of Half-Planes 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Endpoint classification / - Line segment $\mathbf{p} = (\mathbf{p}_x, \mathbf{p}_y) / 0\ 1$ / - Outcodes: / 1001 1000 1010
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Binary Coding of Half-Planes 2/3'?
Page 12	Binary Coding of Half-Planes 3/3
Source cue	- Clipping, if $(oc_0 \& oc_1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip $\mathbf{p}$ at this clip edge
Professor-style explanation	The slide 'Binary Coding of Half-Planes 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Clipping, if $(oc_0 \& oc_1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip $\mathbf{p}$ at this clip edge. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Binary Coding of Half-Planes 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Clipping, if (oc0 & oc1) = 0: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip ■ at this clip edge
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Binary Coding of Half-Planes 3/3'?

Page 13	Pseudo Code
Source cue	<pre>void cohensutherlandLineClipper(Line s, Rectangle r) { int oc0 = outcode(s.p0, r); int oc1 = outcode(s.p1, r); bool done = false; while (!done) { int outcode(Point p, Rectangle r) {</pre>
Professor-style explanation	The slide 'Pseudo Code' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: <pre>void cohensutherlandLineClipper(Line s, Rectangle r) { int oc0 = outcode(s.p0, r); int oc1 = outcode(s.p1, r); bool done = false; while (!done) { int outcode(Point p, Rectangle r) {</pre> The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Pseudo Code. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: <pre>void cohensutherlandLineClipper(Line s, Rectangle r) { int oc0 = outcode(s.p0, r); int oc1 = outcode(s.p1, r); bool done = false; while (!done) { int outcode(Point p, Rectangle r) {</pre>
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?

Page 14	Example
Source cue	1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110

Professor-style explanation	The slide 'Example' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Example. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example' into a causal sentence instead of repeating the slide title?
Page 15	Evaluation
Source cue	- Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside
Professor-style explanation	The slide 'Evaluation' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Evaluation. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?
Page 16	5.2 Cyrus-Beck Line Clipping
Source cue	Clip lines against convex polygon

Professor-style explanation	The slide '5.2 Cyrus-Beck Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Clip lines against convex polygon. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.2 Cyrus-Beck Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Clip lines against convex polygon
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.2 Cyrus-Beck Line Clipping'?

Page 17	Cyrus-Beck Line Clipping
Source cue	- Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky
Professor-style explanation	The slide 'Cyrus-Beck Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Cyrus-Beck Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.

Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Cyrus-Beck Line Clipping'?
Page 18	Parametric Line Representation
Source cue	- Parametric line representation written as follows / $\mathbf{p} = \mathbf{a} + \mathbf{b} \cdot \mathbf{t} - \mathbf{c}$, $0 \leq \mathbf{t} \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 $\mathbf{t}$'s for the 4 clip edges
Professor-style explanation	The slide 'Parametric Line Representation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Parametric line representation written as follows / $\mathbf{p} = \mathbf{a} + \mathbf{b} \cdot \mathbf{t} - \mathbf{c}$, $0 \leq \mathbf{t} \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 $\mathbf{t}$'s for the 4 clip edges. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Parametric Line Representation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Parametric line representation written as follows / $\mathbf{p} = \mathbf{a} + \mathbf{b} \cdot \mathbf{t} - \mathbf{c}$, $0 \leq \mathbf{t} \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 $\mathbf{t}$'s for the 4 clip edges
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Parametric Line Representation'?
Page 19	Intersection Point Calculation 1/2
Source cue	- Consider edge $\mathbf{e}$ and its outward pointing edge normal $\mathbf{n}$ / - Through dot product $\mathbf{p} \cdot (\mathbf{n} - \mathbf{e})$ it can be decided / in which half-space a line point $\mathbf{p}$ lies / $\mathbf{p} \cdot (\mathbf{n} - \mathbf{e}) < 0$ / $\mathbf{p} \cdot (\mathbf{n} - \mathbf{e}) > 0$
Professor-style explanation	The slide 'Intersection Point Calculation 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Consider edge $\mathbf{e}$ and its outward pointing edge normal $\mathbf{n}$ / - Through dot product $\mathbf{p} \cdot (\mathbf{n} - \mathbf{e})$ it can be decided / in which half-space a line point $\mathbf{p}$ lies / $\mathbf{p} \cdot (\mathbf{n} - \mathbf{e}) < 0$ / $\mathbf{p} \cdot (\mathbf{n} - \mathbf{e}) > 0$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Intersection Point Calculation 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Consider edge $\mathbf{e}$ and its outward pointing edge normal $\mathbf{n}$ / - Through dot product $\mathbf{p} \cdot (\mathbf{n}(\mathbf{e}) - \mathbf{e})$ it can be decided / in which half-space a line point $\mathbf{p}(\mathbf{e})$ lies / $\mathbf{p} \cdot (\mathbf{n}(\mathbf{e}) - \mathbf{e}) < 0$ / $\mathbf{p} \cdot \mathbf{n}$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Intersection Point Calculation 1/2'?

Page 20	Intersection Point Calculation 2/2
Source cue	- Determine intersection by calculating $\mathbf{t}$ for $\mathbf{p} \cdot \mathbf{n} - \mathbf{d} = 0$ / - Substitution of $\mathbf{p}$ and regrouping gives $\mathbf{p} \cdot \mathbf{n} - \mathbf{d} / 0$
Professor-style explanation	The slide 'Intersection Point Calculation 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Determine intersection by calculating $\mathbf{t}$ for $\mathbf{p} \cdot \mathbf{n} - \mathbf{d} = 0$ / - Substitution of $\mathbf{p}$ and regrouping gives $\mathbf{p} \cdot \mathbf{n} - \mathbf{d} / 0$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Point Calculation 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Determine intersection by calculating $\mathbf{t}$ for $\mathbf{p} \cdot \mathbf{n} - \mathbf{d} = 0$ / - Substitution of $\mathbf{p}$ and regrouping gives $\mathbf{p} \cdot \mathbf{n} - \mathbf{d} / 0$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Point Calculation 2/2' into a causal sentence instead of repeating the slide title?

Page 21	Determine Relevant t-Values
Source cue	- Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / $\mathbf{p} \cdot \mathbf{n} / \mathbf{p} \cdot \mathbf{n} - \mathbf{d} / \mathbf{p} \cdot \mathbf{n}$
Professor-style explanation	The slide 'Determine Relevant t-Values' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / $\mathbf{p} \cdot \mathbf{n} / \mathbf{p} \cdot \mathbf{n} - \mathbf{d} / \mathbf{p} \cdot \mathbf{n}$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Determine Relevant t-Values. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / $\bullet \bullet \bullet \bullet$ / $\bullet \bullet \bullet \bullet \bullet \bullet$ / $\bullet \bullet$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Determine Relevant t-Values' into a causal sentence instead of repeating the slide title?
Page 22	Pseudo Code
Source cue	<code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code>
Professor-style explanation	The slide 'Pseudo Code' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: <code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code> . The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Pseudo Code. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: <code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code>
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?
Page 23	Axis-Parallel Clipping Region
Source cue	- When clipping region is axis parallel and given by $[x, x] \times [y, y]$, / $\bullet \bullet \bullet \bullet$ $\bullet \bullet \bullet \bullet$ $\bullet \bullet \bullet \bullet$ / following values occur / $\bullet \cdot \bullet - \bullet / 0 \bullet$

Professor-style explanation	The slide 'Axis-Parallel Clipping Region' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - When clipping region is axis parallel and given by $[x_1, x_2] \times [y_1, y_2]$, / $\text{min}(x_1, x_2)$ $\text{min}(y_1, y_2)$ / following values occur / $\text{min}(x_1, x_2) - \text{min}(y_1, y_2) / 0$. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Axis-Parallel Clipping Region. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - When clipping region is axis parallel and given by $[x_1, x_2] \times [y_1, y_2]$, / $\text{min}(x_1, x_2)$ $\text{min}(y_1, y_2)$ / following values occur / $\text{min}(x_1, x_2) - \text{min}(y_1, y_2) / 0$
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Axis-Parallel Clipping Region'?

Page 24	Comparison with Cohen-Sutherland
Source cue	- Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected
Professor-style explanation	The slide 'Comparison with Cohen-Sutherland' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Comparison with Cohen-Sutherland. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Comparison with Cohen-Sutherland'?
Page 25	5.3 Sutherland-Hodgman Polygon Clipping
Source cue	A pipelined version of the scalar product algorithm extended to clip polygons
Professor-style explanation	The slide '5.3 Sutherland-Hodgman Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: A pipelined version of the scalar product algorithm extended to clip polygons. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.3 Sutherland-Hodgman Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: A pipelined version of the scalar product algorithm extended to clip polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.3 Sutherland-Hodgman Polygon Clipping'?
Page 26	Sutherland-Hodgman Polygon Clipping 1/2
Source cue	- Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping
Professor-style explanation	The slide 'Sutherland-Hodgman Polygon Clipping 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 1/2'?

Page 27	Sutherland-Hodgman Polygon Clipping 2/2
Source cue	- Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region
Professor-style explanation	The slide 'Sutherland-Hodgman Polygon Clipping 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 2/2'?

Page 28	Four Cases
Source cue	- Depending on which case occurs, following points added to output list / - Case 1: ■ / - Case 2: Intersection point ■ / - Case 3: -- / - Case 4: Intersection point ■ and ■

Professor-style explanation	The slide 'Four Cases' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Depending on which case occurs, following points added to output list / - Case 1: ■ / - Case 2: Intersection point ■ / - Case 3: -- / - Case 4: Intersection point ■ and ■. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Four Cases. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Depending on which case occurs, following points added to output list / - Case 1: ■ / - Case 2: Intersection point ■ / - Case 3: -- / - Case 4: Intersection point ■ and ■
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Four Cases' into a causal sentence instead of repeating the slide title?

Page 29	Proceeding
Source cue	- Polygon to be clipped is defined by / - Vertices: [■ , ■ , ... , ■] / 1 2 ■ / - Contour: [■ , ■ , ... , ■ , ■] / 1 2 ■ 1
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Polygon to be clipped is defined by / - Vertices: [■ , ■ , ... , ■] / 1 2 ■ / - Contour: [■ , ■ , ... , ■ , ■] / 1 2 ■ 1. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Polygon to be clipped is defined by / - Vertices: [■ , ■ , ... , ■] / 1 2 ■ / - Contour: [■ , ■ , ... , ■ , ■] / 1 2 ■ 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?

Page 30	Pipeline Version
Source cue	- Speed can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware

Professor-style explanation	The slide 'Pipeline Version' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Speed c an be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Pipeline Version. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Speed c an be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pipeline Version'?

Page 31	Pseudo Code
Source cue	<pre>public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {</pre>
Professor-style explanation	The slide 'Pseudo Code' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Pseudo Code. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?
Page 32	Inside Test
Source cue	- Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top
Professor-style explanation	The slide 'Inside Test' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Inside Test. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Inside Test' into a causal sentence instead of repeating the slide title?
Page 33	Intersection Computation
Source cue	- For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y;
Professor-style explanation	The slide 'Intersection Computation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Computation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y;

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Computation' into a causal sentence instead of repeating the slide title?
Page 34	5.4 Weiler-Atherton Polygon Clipping
Source cue	Clip arbitrary polygons against arbitrary clip polygons
Professor-style explanation	The slide '5.4 Weiler-Atherton Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Clip arbitrary polygons against arbitrary clip polygons. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.4 Weiler-Atherton Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Clip arbitrary polygons against arbitrary clip polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.4 Weiler-Atherton Polygon Clipping'?
Page 35	Weiler-Atherton Polygon Clipping
Source cue	- Basic idea: edges of the clip polygon ■ and the polygon ■ to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray)
Professor-style explanation	The slide 'Weiler-Atherton Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Basic idea: edges of the clip polygon ■ and the polygon ■ to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray). The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Weiler-Atherton Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Basic idea: edges of the clip polygon ■ and the polygon ■ to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray)
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Polygon Clipping'?
Page 36	Example Run
Source cue	- Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of ■ and ■ / - Double all edge segments, and label each segment / according to the adjacent region
Professor-style explanation	The slide 'Example Run' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of ■ and ■ / - Double all edge segments, and label each segment / according to the adjacent region. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Example Run. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of ■ and ■ / - Double all edge segments, and label each segment / according to the adjacent region
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example Run' into a causal sentence instead of repeating the slide title?
Page 37	Intersection Types
Source cue	- There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal)

Professor-style explanation	The slide 'Intersection Types' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Types. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Types' into a causal sentence instead of repeating the slide title?

Page 38	Contour Processing
Source cue	- In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of ■ / and insert the contour edges of ■ one after the other
Professor-style explanation	The slide 'Contour Processing' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of ■ / and insert the contour edges of ■ one after the other. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Contour Processing. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of ■ / and insert the contour edges of ■ one after the other
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Contour Processing' into a causal sentence instead of repeating the slide title?

Page 39	Evaluation
Source cue	- Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion
Professor-style explanation	The slide 'Evaluation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Evaluation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?
Page 40	5.5 Greiner-Hormann Polygon Clipping
Source cue	Exploitation of the winding number to clip arbitrary polygons
Professor-style explanation	The slide '5.5 Greiner-Hormann Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Exploitation of the winding number to clip arbitrary polygons. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.5 Greiner-Hormann Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Exploitation of the winding number to clip arbitrary polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.5 Greiner-Hormann Polygon Clipping'?
Page 41	Greiner-Hormann

Source cue	- Basic Idea: clipped region is made up of all edges of ■ that lie in ■, / combined with all edges of ■ that lie in ■ / [Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Greiner-Hormann' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Basic Idea: clipped region is made up of all edges of ■ that lie in ■, / combined with all edges of ■ that lie in ■ / [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Greiner-Hormann. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Basic Idea: clipped region is made up of all edges of ■ that lie in ■, / combined with all edges of ■ that lie in ■ / [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Greiner-Hormann'?
Page 42	Chalk Wagon Metaphor
Source cue	- Using a chalk wagon metaphor / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■ / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■
Professor-style explanation	The slide 'Chalk Wagon Metaphor' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Using a chalk wagon metaphor / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■ / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Chalk Wagon Metaphor. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Using a chalk wagon metaphor / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■ / - Drive along all edges of ■ with a chalk wagon / that is only open when the current position is within ■

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Chalk Wagon Metaphor' into a causal sentence instead of repeating the slide title?
Page 43	Contour Detection
Source cue	- The edges of polygon ■ within polygon ■ can thus be found as follows / - Insert in ■ all intersection points with ■ / - Start at any vertex ■ of ■ / - If ■ is within ■, the edge flag is turned on, otherwise not / - Traverse edges of ■ and invert edge marker flag when intersecting with ■
Professor-style explanation	The slide 'Contour Detection' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - The edges of polygon ■ within polygon ■ can thus be found as follows / - Insert in ■ all intersection points with ■ / - Start at any vertex ■ of ■ / - If ■ is within ■, the edge flag is turned on, otherwise not / - Traverse edges of ■ and invert edge marker flag when intersecting with ■. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Contour Detection. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - The edges of polygon ■ within polygon ■ can thus be found as follows / - Insert in ■ all intersection points with ■ / - Start at any vertex ■ of ■ / - If ■ is within ■, the edge flag is turned on, otherwise not / - Traverse edges of ■ and invert edge marker flag when intersecting with ■
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Contour Detection' into a causal sentence instead of repeating the slide title?
Page 44	Set Operations
Source cue	[Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Set Operations' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Set Operations. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Set Operations'?

Page 45	Data Structure 1/2
Source cue	- Polygons exist as a closed, double-linked list / ■ , ■ , ■ , ... , ■ = ■ / 0 1 2 ■ 0 / - The following segments are included / ■ ■ , ■ ■ , ... , ■ ■ = ■ ■
Professor-style explanation	The slide 'Data Structure 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Polygons exist as a closed, double-linked list / ■ , ■ , ■ , ... , ■ = ■ / 0 1 2 ■ 0 / - The following segments are included / ■ ■ , ■ ■ , ... , ■ ■ = ■ ■. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Data Structure 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Polygons exist as a closed, double-linked list / ■ , ■ , ■ , ... , ■ = ■ / 0 1 2 ■ 0 / - The following segments are included / ■ ■ , ■ ■ , ... , ■ ■ = ■ ■
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Data Structure 1/2' into a causal sentence instead of repeating the slide title?

Page 46	Data Structure 2/2
Source cue	[Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Data Structure 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Data Structure 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: [Greiner & Hormann, TOG 1998]

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Data Structure 2/2'?
Page 47	Proceeding
Source cue	- Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule)
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?
Page 48	Pseudo Code
Source cue	while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current)
Professor-style explanation	The slide 'Pseudo Code' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Pseudo Code. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?

Page 49	Winding Number 1/3
Source cue	- Winding number of a point ■ and a closed curve ■ / indicates the number of times a ray that traverses the entire curve from ■ / winds around ■ / ■ ■, ■ = (cid:3505) ■■ / - Counterclockwise: +1
Professor-style explanation	The slide 'Winding Number 1/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Winding number of a point ■ and a closed curve ■ / indicates the number of times a ray that traverses the entire curve from ■ / winds around ■ / ■ ■, ■ = (cid:3505) ■■ / - Counterclockwise: +1. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Winding Number 1/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Winding number of a point ■ and a closed curve ■ / indicates the number of times a ray that traverses the entire curve from ■ / winds around ■ / ■ ■, ■ = (cid:3505) ■■ / - Counterclockwise: +1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Winding Number 1/3' works per object, per image region, per ray, or per fragment?

Page 50	Winding Number 2/3
Source cue	- The winding number of a point ■ has the following useful properties / 1. If ■ or the curve ■ is continuously changed so that ■ maintains a / positive distance to ■, the winding number will not change / For curve ■, the winding number is constant in each region formed by ■ / 2. When ■ moves and ■ crosses once, winding number changes by exactly 1

Professor-style explanation	The slide 'Winding Number 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - The winding number of a point ■ has the following useful properties / 1. If ■ or the curve ■ is continuously changed so that ■ maintains a / positive distance to ■, the winding number will not change / For curve ■, the winding number is constant in each region formed by ■ / 2. When ■ moves and ■ crosses once, winding number changes by exactly 1. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Winding Number 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - The winding number of a point ■ has the following useful properties / 1. If ■ or the curve ■ is continuously changed so that ■ maintains a / positive distance to ■, the winding number will not change / For curve ■, the winding number is constant in each region formed by ■ / 2. When ■ moves and ■ crosses once, winding number changes by exactly 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Winding Number 2/3' into a causal sentence instead of repeating the slide title?
Page 51	Winding Number 3/3
Source cue	- From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Winding Number 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Winding Number 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Winding Number 3/3'?
Page 52	Special Cases
Source cue	- Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge
Professor-style explanation	The slide 'Special Cases' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Special Cases. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Special Cases' into a causal sentence instead of repeating the slide title?
Page 53	Evaluation
Source cue	- Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,
Professor-style explanation	The slide 'Evaluation' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Evaluation. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?
Page 54	by exploiting binary outcodes
Source cue	- Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon
Professor-style explanation	The slide 'by exploiting binary outcodes' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about by exploiting binary outcodes. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'by exploiting binary outcodes'?
Page 55	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?
Page 56	Addison-Wesley 2013.
Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016.
Professor-style explanation	The slide 'Addison-Wesley 2013.' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Addison-Wesley 2013.. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Addison-Wesley 2013.' into a causal sentence instead of repeating the slide title?

05.1 Cohen-Sutherland Line Clipping

Source location: Section 5.1

Commentary

Cohen-Sutherland assigns outcodes to line endpoints based on which side of the clipping window they lie on. These codes allow quick accept, quick reject, or partial clipping. The algorithm is efficient because many cases are decided before computing intersections.

The exam-friendly idea is the logic of region codes: if both endpoints are inside, keep the segment. If the bitwise AND of the outcodes is nonzero, both endpoints share an outside region and the segment can be rejected. Otherwise, compute an intersection and continue.

Mental model

Outcodes are a cheap classification before doing intersection math.

Check yourself

What does a nonzero bitwise AND of two endpoint outcodes imply?

05.2 Cyrus-Beck Line Clipping

Source location: Section 5.2

Commentary

Cyrus-Beck treats the line parametrically and clips against convex boundaries. Instead of repeatedly using region codes, it finds entering and leaving parameter values along the line.

The key is to reason about the line parameter t . A clipped line segment is not a new unrelated line; it is a restricted interval of the original parametric line. Boundary tests shrink that interval.

Mental model

Clipping a parametric line means narrowing the valid t interval.

Check yourself

Why does Cyrus-Beck naturally require convex clipping regions?

05.3 Sutherland-Hodgman Polygon Clipping

Source location: Section 5.3

Commentary

Sutherland-Hodgman clips a polygon against one boundary at a time. For each edge of the polygon, it decides whether vertices are inside or outside and emits zero, one, or two vertices depending on transitions across the boundary.

This algorithm is easiest to understand as a stream processor. Feed in a polygon, clip it against the left boundary, feed the result into the right boundary, and so on. Each boundary can add intersection vertices.

Mental model

Polygon clipping is repeated edge-by-edge filtering plus intersection insertion.

Check yourself

When can polygon clipping increase the number of vertices?

05.4 Weiler-Atherton and Greiner-Hormann

Source location: Sections 5.4-5.5

Commentary

These algorithms address more complex polygon clipping scenarios, especially when polygon relationships are not as simple as convex-window clipping. They organize intersections and traversal rules to produce correct output boundaries.

For preparation, focus on why simpler algorithms are not enough. Complex polygon intersection can require following alternating boundaries of subject and clipping polygons. That is a topology problem, not only a line-intersection problem.

Mental model

Complex polygon clipping is about traversing boundary networks after intersections are known.

Check yourself

Why are intersection ordering and traversal rules important for polygon clipping?

End-of-lecture summary

If you remember only one thing from Lecture 05, remember this: This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.